



Department of
Computer Science
& Applied Physics

H.Dip. in Science (Software Development) - Object-Oriented
Software Development (2025 – 26)

Encrypting Files with a Four Square Cipher (35%)

ASSIGNMENT DESCRIPTION & SCHEDULE

YOU MAY NOT USE ANY GENERATIVE AI IN THIS ASSIGNMENT

Overview

You are required to develop a Java application that is capable of encrypting and decrypting a text file or URL using a *four-square cipher*. The four-square cipher was invented by the French cryptographer Felix Delastelle (1840-1902) and uses four 5x5 matrices arranged in a square to encrypt pairs of characters in a message. Each of the 5x5 matrices contains 25 letters (the relatively rare letter *j* is merged with the letter *i*). The upper-left and lower-right quadrants (shown in green) are *plaintext squares* containing a standard alphabet. The upper-right and lower-left quadrants (shown in blue) are the *ciphertext squares* contain a mixed random alphabetic sequence. The *four-square cipher* is a polygraphic substitution cipher that uses the same keyword to encrypt plain-text and decrypt cipher-text, i.e. the key is symmetric.

A	B	C	D	E	Z	G	P	T	F
F	G	H	I	K	O	I	H	M	U
L	M	N	O	P	W	D	R	C	N
Q	R	S	T	U	Y	K	E	Q	A
V	W	X	Y	Z	X	V	S	B	L
M	F	N	B	D	A	B	C	D	E
C	R	H	S	A	F	G	H	I	K
X	Y	O	G	V	L	M	N	O	P
I	T	U	E	W	Q	R	S	T	U
L	Q	Z	K	P	V	W	X	Y	Z

The encryption key is used to generate a unique sequence of letters that can be used to populate the two *ciphertext squares*. The key can be randomly generated, specified with one keyword or specified using two different keywords (dropping duplicate letters). Any remaining spaces, i.e. a key length < 25 characters, should be filled with the remaining letters of the alphabet in order.

Minimum Requirements

You are required to develop a Java application that can parse, encrypt and decrypt the contents of a text file (small or very large...) or URL using the four-square cipher.

- The application should contain a command-line menu that prompts the user to enter a keyword(s) and a file or URL. It should then parse the given resource line-by-line, encrypt / decrypt the text and then append to an output file.
- You must use a 2D array structure to implement the four-square cipher. Do not use any of the data structures in the *Java Collections Framework*. The knowledge you will gain from mastering array structures will prove invaluable for the programming modules in the next semester.

The objective of the assignment is to reinforce the programming concepts that we have covered since the start of the semester. In addition to the methods above, you are free to add any additional functionality you want. The application of good programming practice and object-oriented techniques will be rewarded.

Deployment and Submission

Please **read the following carefully** and pay particular attention to the files that you are required to submit and those that you should not include with your assignment:

- ***The project must be submitted by 5pm on Sunday January 25th 2026.*** Before submitting the assignment, you should review and test it from a command prompt on a different computer to the one that you used to program the project.
- The project must be submitted as a Zip archive (***not a 7z, rar or WinRar file***) using the Moodle upload utility. You can find the area to upload the project under the “*Encoding Words with Suffixes - (50%) Assignment Upload*” link on Moodle. Only the contents of the submitted Zip will be considered. **Do not add comments to the Moodle assignment upload form.**
- The name of the Zip archive should be `<id>.zip` where `<id>` is your ATU student number.

Do NOT submit the assignment as an Eclipse project or submit any text files or other resources. If you do, you will lose marks.
You will also lose marks if you hard-code any environmental variables in your project like system paths and file names.

- The Zip archive should have the structure shown below.

Item	Description
src	A directory that contains the packaged source code for your application.
README.pdf	A PDF file detailing the main features of your application in no more than 300 words . Marks will only be given for features that are described in the README.

Scoring Rubric

Component	Marks	Description
<i>Robustness & Design</i>	50	The application deploys and executes correctly.
<i>Documentation</i>	50	Comments and supporting documentation.

How to Encrypt and Decrypt a Four-Square Cipher

Encryption and decryption are implemented in the following steps:

- **Step 1: Set the Keyword(s)**

Either generate or request a keyword with a length ≥ 50 or two keywords with a length ≥ 25 containing unique characters.

Top Right Quadrant: ZGPTFOIHMUWDRCNKYKEQAXVSBL

Bottom Left Quadrant: MFNBDCRHSAXYOGVITUEWLQZKP

Populate the top right and bottom left quadrants with the letters from the keyword(s).

- **Step 2: Read in the Plaintext**

Read in the plaintext from some source and convert it to either upper or lowercase to match the case of the matrix characters:

THECURFEWTOLLSTHEKNELLOFPARTINGDAY

- **Step 3: Break up the Plaintext into Bigrams**

Plaintext: THECURFEWTOLLSTHEKNELLOFPARTINGDAY

Bigrams: TH EC UR FE WT OL LS TH EK NE LL OF PA RT IN GD AY

An 'X' (or other character) can be appended to the end to ensure an even length.

- **Step 4: Encrypt Bigram**

Locate each of the two characters in the bigram in the top left and the bottom right square respectively. Then encrypt each plain character as follows, e.g. {TH} $\rightarrow$ {ES}

A	B	C	D	E	Z	G	P	T	F
F	G	H	I	K	O	I	H	M	U
L	M	N	O	P	W	D	R	C	N
Q	R	S	T	U	Y	K	E	Q	A
V	W	X	Y	Z	X	V	S	B	L
M	F	N	B	D	A	B	C	D	E
C	R	H	S	A	F	G	H	I	K
X	Y	O	G	V	L	M	N	O	P
I	T	U	E	W	Q	R	S	T	U
L	Q	Z	K	P	V	W	X	Y	Z

- *Char 1:* In the top-right square, find the character at the intersection of the row of the plaintext character in the top left square with the column of the plaintext character in the bottom right square.
- *Char 2:* In the bottom-left square, find the character at the intersection of the column of the plaintext character in the top left square with the row of the plaintext character in the bottom right square.

Ciphertext sequence: ESPDKWUMBTWGRIESFANNWXSWDQTHGMFTL

- **Step 5: Decrypt Bigram**

Locate each of the two characters in the bigram in the top right and the bottom left square respectively. Then decrypt each plain character as follows, e.g. e.g.

{ES} $\rightarrow$ {TH}:

A	B	C	D	E	Z	G	P	T	F
F	G	H	I	K	O	I	H	M	U
L	M	N	O	P	W	D	R	C	N
Q	R	S	T	U	Y	K	E	Q	A
V	W	X	Y	Z	X	V	S	B	L
M	F	N	B	D	A	B	C	D	E
C	R	H	S	A	F	G	H	I	K
X	Y	O	G	V	L	M	N	O	P
I	T	U	E	W	Q	R	S	T	U
L	Q	Z	K	P	V	W	X	Y	Z

- **Char 1:** In the top-left square, find the character at the intersection of the row of the ciphertext character in the top right square with the column of the ciphertext character in the bottom left square.
- **Char 2:** In the bottom-right square, find the character at the intersection of the column of the ciphertext character in the top right square with the row of the plaintext ciphertext in the bottom left square.